

Object-Oriented Programming

Unit 4: MVC Pattern, Command Pattern, Exception Handling, Packages & Memory Management

MAKAUT B.Tech IT | Exam-Ready Notes with Java Code

1. Model-View-Controller (MVC) Pattern

MVC is a **structural/architectural design pattern** that divides an application into three separate, independently manageable components. Its primary purpose is **separation of concerns** — each component has one responsibility and changes to one do not ripple across the others.

Component	Responsibility	Real-world analogy
Model	Manages data, business logic, and application state	The database or business rules of a banking app
View	Presentation layer — displays data from Model to user	ATM screen showing your balance
Controller	Intermediary between Model and View. Handles user input and updates Model accordingly	ATM keypad/button which sends data to the database

Communication flow:

- User interacts with View (clicks button, submits form)
- View forwards the input to Controller
- Controller processes it, updates Model accordingly
- Model notifies View of state change
- View requests fresh data from Model and re-renders the UI

Complete MVC Implementation — Student Management System:

```
// ■■■ MODEL ■■■ manages data and business logic
class Student {
    private String name;
    private String rollNo;

    public Student(String name, String rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }

    public String getName() { return name; }
    public void setName(String n) { this.name = n; }
    public String getRollNo() { return rollNo; }
    public void setRollNo(String r) { this.rollNo = r; }
}

// ■■■ VIEW ■■■ presentation only; no business logic
class StudentView {
    public void printStudentDetails(String name, String rollNo) {
        System.out.println("=== Student Details ===");
        System.out.println("Name : " + name);
        System.out.println("Roll No: " + rollNo);
    }
}
```

```

}

// ■■■ CONTROLLER ■■■ intermediary; coordinates Model <-> View
class StudentController {
    private Student    model;
    private StudentView view;

    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
    }

    // Delegate getters/setters to Model
    public String getStudentName()          { return model.getName(); }
    public void   setStudentName(String n)  { model.setName(n); }
    public String getStudentRollNo()        { return model.getRollNo(); }
    public void   setStudentRollNo(String r) { model.setRollNo(r); }

    // Trigger View update
    public void updateView() {
        view.printStudentDetails(model.getName(), model.getRollNo());
    }
}

// ■■■ MAIN / Client ■■■
class Main {
    // Simulates fetching data from a database
    private static Student fetchFromDB() {
        return new Student("Abhilash", "101");
    }

    public static void main(String[] args) {
        Student    model    = fetchFromDB();
        StudentView view    = new StudentView();
        StudentController controller = new StudentController(model, view);

        controller.updateView();          // display initial state

        controller.setStudentName("Riya"); // simulate user update
        controller.updateView();          // display updated state
    }
}

// Output:
// === Student Details ===
// Name    : Abhilash
// Roll No: 101
// === Student Details ===
// Name    : Riya
// Roll No: 101

```

Advantage	Explanation
Separation of concerns	UI, logic, and data are completely decoupled
Parallel development	UI team and backend team can work simultaneously
Easier testing	Model can be unit-tested without any UI code
Reusability	Same Model can serve different Views (web + mobile)

Maintainability	Change the View without touching business logic, and vice versa
-----------------	---

2. Commands as Methods and as Objects

A **command** is an instruction that tells the program to perform an action. In Java, a command can be expressed in two fundamentally different ways, and understanding both is what your syllabus calls "commands as methods and as objects".

2.1 Commands as Methods

The simplest form — a method is a named block of code that performs a specific action when invoked. The command is embedded directly in the method definition; it executes immediately when called and cannot be stored, queued, or passed around.

```
class Light {
    // Each method IS a command
    public void turnOn() { System.out.println("Light ON"); }
    public void turnOff() { System.out.println("Light OFF"); }
    public void dim(int level) {
        System.out.println("Light dimmed to " + level + "%");
    }
}

class Main {
    public static void main(String[] args) {
        Light light = new Light();
        light.turnOn();      // command executed immediately
        light.dim(50);
        light.turnOff();
    }
}
```

Limitation: You can't delay execution, undo it, log it, queue it, or pass the command itself to another object. That's where Command-as-Object comes in.

2.2 Commands as Objects — The Command Pattern

The **Command Pattern** is a behavioral design pattern that **encapsulates a request as an object**. This lets you parameterise other objects with commands, queue or log requests, and support undo/redo operations.

Key components:

Component	Role
Command interface	Declares execute() (and optionally undo())
ConcreteCommand	Implements Command; holds reference to Receiver; calls its action in execute()
Receiver	The object that actually performs the action (Light, Fan, etc.)
Invoker	Holds a Command reference; calls execute() — doesn't know what the command does
Client	Creates ConcreteCommand objects and wires them to the Invoker

```
// ■■ Command interface
interface Command {
    void execute();
    void undo();      // optional but common in exam questions
}

// ■■ Receiver – performs the actual work
class Light {
```

```

    private String name;
    Light(String name) { this.name = name; }
    void on() { System.out.println(name + " Light ON"); }
    void off() { System.out.println(name + " Light OFF"); }
}

class Fan {
    void start() { System.out.println("Fan started"); }
    void stop() { System.out.println("Fan stopped"); }
}

// ■■ ConcreteCommands
class LightOnCommand implements Command {
    private Light light;
    LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.on(); }
    public void undo() { light.off(); }
}

class LightOffCommand implements Command {
    private Light light;
    LightOffCommand(Light light) { this.light = light; }
    public void execute() { light.off(); }
    public void undo() { light.on(); }
}

class FanStartCommand implements Command {
    private Fan fan;
    FanStartCommand(Fan fan) { this.fan = fan; }
    public void execute() { fan.start(); }
    public void undo() { fan.stop(); }
}

// ■■ Invoker – a remote control with history for undo
class RemoteControl {
    private Command lastCommand;

    public void pressButton(Command cmd) {
        cmd.execute();
        lastCommand = cmd;
    }

    public void pressUndo() {
        if (lastCommand != null) {
            lastCommand.undo();
            lastCommand = null;
        }
    }
}

// ■■ Client
class Main {
    public static void main(String[] args) {
        Light bedroom = new Light("Bedroom");
        Fan fan = new Fan();

        Command lightOn = new LightOnCommand(bedroom);
        Command lightOff = new LightOffCommand(bedroom);
        Command fanStart = new FanStartCommand(fan);
    }
}

```

```

RemoteControl remote = new RemoteControl();

remote.pressButton(lightOn);    // Bedroom Light ON
remote.pressButton(fanStart);   // Fan started
remote.pressUndo();             // Fan stopped (undo last)
remote.pressButton(lightOff);   // Bedroom Light OFF
remote.pressUndo();             // Bedroom Light ON (undo)
}

```

Feature	Commands as Methods	Commands as Objects
Execution	Immediate, in-place	Can be deferred, queued, logged
Undo support	Not possible	undo() method on Command object
Parameterisable	No	Yes — pass Command to any Invoker
Flexibility	Low	High — swap commands at runtime
Complexity	Simple	More classes, but far more powerful
Use when	Simple, one-off actions	GUI buttons, macro recording, undo/redo, job queues

3. Exception Handling in Java

An **exception** is an unexpected event that disrupts the normal flow of a program at runtime — for example, dividing by zero, accessing a null reference, or reading a file that doesn't exist. Java's exception handling mechanism lets you **detect, respond to, and recover from** these events gracefully instead of crashing.

3.1 Exception Hierarchy

All exception/error types form a class hierarchy rooted at `java.lang.Throwable`:

```
Throwable
■ ■ ■ Error (serious JVM problems – don't catch these)
■ ■ ■ OutOfMemoryError
■ ■ ■ StackOverflowError
■ ■ ■ VirtualMachineError
■ ■ ■ Exception (things programs should handle)
■ ■ ■ IOException ← Checked exception
■ ■ ■ SQLException ← Checked exception
■ ■ ■ ClassNotFoundException ← Checked exception
■ ■ ■ RuntimeException ← Unchecked exception (root)
■ ■ ■ ArithmeticException (divide by zero)
■ ■ ■ NullPointerException (null reference access)
■ ■ ■ ArrayIndexOutOfBoundsException
■ ■ ■ ClassCastException
■ ■ ■ NumberFormatException
■ ■ ■ IllegalArgumentException
```

Type	When detected	Must handle?	Examples
Checked Exception	Compile time	YES — compiler forces you to handle it	<code>IOException</code> , <code>SQLException</code> , <code>FileNotFoundException</code>
Unchecked Exception (RuntimeException)	Runtime	NO — optional but recommended	<code>NullPointerException</code> , <code>ArithmeticException</code> , <code>ArrayIndexOutOfBoundsException</code>
Error	Runtime (JVM level)	NO — don't catch; irrecoverable	<code>OutOfMemoryError</code> , <code>StackOverflowError</code>

3.2 Exception Handling Keywords

Keyword	Purpose
try	Wraps code that might throw an exception
catch	Handles a specific exception type thrown by the try block
finally	Always executes — whether exception occurred or not; used for cleanup
throw	Manually throw a specific exception object
throws	Declares in a method signature that it may throw a checked exception

3.3 try-catch-finally (full syntax):

```
class ExceptionDemo {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30};

        try {
            System.out.println("Trying...");
            int result = arr[5]; // ArrayIndexOutOfBoundsException
        }
    }
}
```

```

        System.out.println(result);    // never reached
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Caught: " + e.getMessage());
    } catch (Exception e) {
        // catches any other Exception – must come AFTER specific ones
        System.out.println("General exception: " + e.getMessage());
    } finally {
        // ALWAYS runs – even if no exception, or even if catch re-throws
        System.out.println("Finally block executed – cleanup here");
    }

    System.out.println("Program continues normally after handled exception");
}
// Output:
// Trying...
// Caught: Index 5 out of bounds for length 3
// Finally block executed – cleanup here
// Program continues normally after handled exception

```

Important: Multiple catch blocks must go from most specific to most general — if you put Exception first, the compiler will complain that subsequent catches are unreachable.

3.4 throw vs throws

```

// throw – manually throw an exception inside a method body
class Validator {
    static void validateAge(int age) {
        if (age < 0 || age > 150) {
            throw new IllegalArgumentException("Invalid age: " + age);
        }
        System.out.println("Age is valid: " + age);
    }
}

// throws – declare a checked exception in the method signature
import java.io.*;
class FileReader {
    static void readFile(String path) throws IOException {
        // compiler requires throws because IOException is checked
        java.io.FileReader fr = new java.io.FileReader(path);
        System.out.println("File opened successfully");
        fr.close();
    }
}

class Main {
    public static void main(String[] args) {
        // throw example
        try {
            Validator.validateAge(-5);
        } catch (IllegalArgumentException e) {
            System.out.println("Caught: " + e.getMessage());
        }

        // throws example – caller must handle the checked exception
        try {
            FileReader.readFile("data.txt");
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}

```

```

    }
}

```

3.5 Custom (User-Defined) Exceptions

You can create your own exception class by extending `Exception` (for checked) or `RuntimeException` (for unchecked). This is a very common exam question.

```

// Custom checked exception
class InsufficientFundsException extends Exception {
    private double amount;

    public InsufficientFundsException(double amount) {
        super("Insufficient funds. Short by: " + amount);
        this.amount = amount;
    }

    public double getAmount() { return amount; }
}

// Custom unchecked exception
class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance;

    BankAccount(double balance) { this.balance = balance; }

    // throws declares checked custom exception
    void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException(amount - balance);
        }
        balance -= amount;
        System.out.println("Withdrawn: " + amount + "    Balance: " + balance);
    }
}

class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(500);

        try {
            acc.withdraw(200);    // OK
            acc.withdraw(400);    // throws InsufficientFundsException
        } catch (InsufficientFundsException e) {
            System.out.println("Error: " + e.getMessage());
            System.out.println("Short by: " + e.getAmount());
        } finally {
            System.out.println("Transaction attempt complete.");
        }

        // Unchecked custom exception – no try/catch required by compiler
        try {
            int age = -5;
            if (age < 0) throw new InvalidAgeException("Age cannot be negative: " + age);
        }
    }
}

```

```

        } catch (InvalidAgeException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}

```

3.6 Multi-catch and try-with-resources (Java 7+)

```

// Multi-catch: handle multiple types in one catch block (Java 7+)
try {
    // risky code
} catch (IOException | SQLException e) {
    System.out.println("IO or SQL error: " + e.getMessage());
}

// try-with-resources: auto-closes resource after block exits
// The class must implement AutoCloseable / Closeable
import java.io.*;
try (FileReader fr = new FileReader("data.txt");
     BufferedReader br = new BufferedReader(fr)) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
// fr and br are automatically closed here – no finally needed

```

4. Packages in Java

A **package** is a namespace that organises related classes and interfaces into groups, preventing naming conflicts between classes from different sources and controlling access. Think of it as a folder in a file system for your Java classes.

Why packages?

- **Name collision avoidance** — two classes named Date can coexist in java.util.Date and java.sql.Date.
- **Access control** — the default access modifier means "accessible within the same package only".
- **Organisation** — large projects group related classes logically.
- **Reusability** — packaged code is easily imported and reused.

Types of packages:

Type	Description	Examples
Built-in / Java API packages	Provided by the JDK; ready to import	java.lang (auto-imported), java.util, java.io, java.net, java
User-defined packages	Created by the developer using the package keyword	com.company.project, in.nsec.oop

Creating and using a user-defined package:

```
// File: com/nsec/oop/MathUtils.java
package com.nsec.oop; // MUST be the first statement in the file

public class MathUtils {
    public static int add(int a, int b) { return a + b; }
    public static int multiply(int a, int b) { return a * b; }

    // package-private – accessible only within com.nsec.oop
    static void helper() {
        System.out.println("Internal helper");
    }
}

// File: Main.java (in a different package or default package)
import com.nsec.oop.MathUtils; // import specific class
// import com.nsec.oop.*; // import all classes from package

class Main {
    public static void main(String[] args) {
        System.out.println(MathUtils.add(3, 4)); // 7
        System.out.println(MathUtils.multiply(3, 4)); // 12
        // MathUtils.helper(); // ERROR – package-private, not accessible here
    }
}
```

Compile and run with packages:

```
javac -d . com/nsec/oop/MathUtils.java (creates folder structure)
javac Main.java
java Main
```

Access modifiers + packages — full matrix:

Modifier	Same class	Same package	Subclass (diff pkg)	Other package
----------	------------	--------------	---------------------	---------------

private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Static imports:

```
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;

class Circle {
    double area(double r) {
        return PI * r * r;    // no Math.PI needed
    }
    double diagonal(double a, double b) {
        return sqrt(a*a + b*b); // no Math.sqrt
    }
}
```

5. Implementing OO Language Features

This section covers how Java implements the key mechanics that make OOP work at the language level — features you use all the time but may not have formally studied.

5.1 Static Members

A static member belongs to the **class itself**, not to any individual object. All objects share the same copy. Accessed via `ClassName.member`.

```
class Counter {
    private static int count = 0; // shared across ALL instances
    private int id;

    Counter() {
        count++;
        id = count;
    }

    static int getCount() { return count; } // static method
    int getId()           { return id; }    // instance method

    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();
        System.out.println("Total: " + Counter.getCount()); // 3
        System.out.println("c2 id: " + c2.getId());          // 2
    }
}
```

Static methods cannot use this or instance variables — they have no object context. This is why `main()` is static: the JVM calls it before any object exists.

5.2 Nested and Inner Classes

Type	Declared	Can access outer class members	Use case
Static nested class	static inside outer class	Only static members	Utility/helper logically tied to outer class
Non-static inner class	inside outer class, no static	All (including private)	Needs access to outer class state
Local class	inside a method	Method's local variables (final/finally)	One-off helper, defined only in that method
Anonymous class	As inline expression	All outer members	Quick one-time interface/abstract class implementation

```
class Outer {
    private int x = 10;

    // Non-static inner class
    class Inner {
        void display() {
            System.out.println("Outer x = " + x); // accesses outer's private field
        }
    }

    // Static nested class
    static class StaticNested {
        void greet() { System.out.println("Static nested class"); }
        // Cannot access x – no outer instance
    }
}
```

```

}

// Anonymous class – implement interface inline
interface Greeting {
    void greet(String name);
}

class Main {
    public static void main(String[] args) {
        // Inner class usage
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();
        inner.display();                // Outer x = 10

        // Static nested class usage
        Outer.StaticNested sn = new Outer.StaticNested();
        sn.greet();

        // Anonymous class – define and instantiate in one step
        Greeting g = new Greeting() {
            @Override
            public void greet(String name) {
                System.out.println("Hello, " + name + "!");
            }
        };
        g.greet("Abhilash");            // Hello, Abhilash!
    }
}

```

5.3 Enumerations (enum)

An enum is a special class that represents a fixed set of named constants. Much more powerful than integer constants — type-safe, can have fields and methods.

```

enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY;

    public boolean isWeekend() {
        return this == SATURDAY || this == SUNDAY;
    }
}

enum Planet {
    MERCURY(3.303e+23, 2.4397e6),
    EARTH (5.976e+24, 6.37814e6);

    private final double mass;
    private final double radius;
    static final double G = 6.67300E-11;

    Planet(double mass, double radius) {
        this.mass = mass;
        this.radius = radius;
    }
    double surfaceGravity() { return G * mass / (radius * radius); }
}

class Main {
    public static void main(String[] args) {
        Day today = Day.SATURDAY;
        System.out.println(today + " weekend? " + today.isWeekend()); // SATURDAY weekend?
    }
}

```

```
true

    for (Day d : Day.values()) {
        if (!d.isWeekend()) System.out.println("Workday: " + d);
    }

    System.out.printf("Earth gravity: %.2f%n", Planet.EARTH.surfaceGravity());
}
```

6. Memory Management in Java

Java manages memory automatically through the **Java Virtual Machine (JVM)** and its **Garbage Collector (GC)**. Unlike C/C++, the developer does not need to manually call `free()` or `delete` — the JVM tracks object lifetimes and reclaims memory when objects are no longer reachable.

6.1 JVM Memory Areas

Memory Area	What it stores	Managed by	Characteristics
Stack	Method call frames, local variables, JVM runtime data (not objects themselves), local results	JVM (automatic)	Thread-private, grows down stack; fast allocation/deallocation
Heap	All objects created with <code>new</code> ; instantiated by <code>Garbage Collector</code>	Garbage Collector	Shared across all threads; larger but slower than stack
Method Area (Metaspace)	Class definitions, static variables, JVM metadata	JVM (class loading)	Shared across all threads; renamed Metaspace in Java 8
PC Registers	Address of currently executing JVM instruction	JVM	One per thread; not directly programmer-accessible
Native Method Stack	Native (non-Java) method calls	JVM	Used for JNI (Java Native Interface) calls

Stack vs Heap — the most important distinction for exams:

```
class Person {
    String name;      // instance variable – on HEAP (inside the object)
    int    age;        // instance variable – on HEAP
    Person(String name, int age) { this.name = name; this.age = age; }
}

class Main {
    public static void main(String[] args) {
        // 'args' → reference on STACK → points to array on HEAP
        // 'p'     → reference on STACK → points to Person object on HEAP
        // 'score' → primitive on STACK (value stored directly, not on heap)
        Person p    = new Person("Abhilash", 20);
        int    score = 95;

        System.out.println(p.name + " scored " + score);

        // When main() returns:
        // Stack frame for main() is POPPED and destroyed
        // 'p' reference is gone → Person object on heap is now UNREACHABLE
        // GC will eventually reclaim that heap memory
    }
}
```

6.2 Garbage Collection (GC)

The **Garbage Collector** is a JVM background thread that automatically identifies and frees **unreachable objects** on the heap — objects that no running thread can reach through any chain of references.

GC lifecycle:

- **Object creation** — `new Person()` allocates memory on the heap.
- **Object in use** — while any reference points to it, the object is **reachable** and won't be collected.
- **Object becomes unreachable** — when all references to it are gone (go out of scope, set to null, overwritten), it becomes eligible for GC.

- **Finalization** — GC calls `finalize()` on the object before reclaiming memory (deprecated in Java 9+; don't rely on it).
- **Memory reclaimed** — heap memory is freed and returned to the pool.

```
class Demo {
    static void createObjects() {
        Person p1 = new Person("Alice", 25);    // p1 on stack → object on heap
        Person p2 = new Person("Bob", 30);      // p2 on stack → object on heap
        p1 = p2;    // Alice object has no reference → eligible for GC
        p2 = null;  // p2 cleared; Bob object still referenced by p1
    }    // method returns: p1 goes out of scope → Bob object also eligible for GC

    public static void main(String[] args) {
        createObjects();

        // Suggest GC run (just a hint – JVM may ignore it)
        System.gc();

        // Make an object eligible explicitly
        Person p = new Person("Carol", 22);
        System.out.println(p.name);
        p = null;    // Carol object now eligible for GC
        // System.gc();
    }
}
```

6.3 Memory Leaks in Java

Java can still have memory leaks when objects are **referenced but no longer needed**. The GC cannot collect them because a reference still exists, even though the program doesn't use them anymore.

- **Static collections** — objects added to a static list/map and never removed live for the entire program lifetime.
- **Unclosed resources** — file handles, database connections, sockets not closed in a finally block or try-with-resources.
- **Inner class holding outer reference** — non-static inner class always holds an implicit reference to its outer class instance.
- **Listeners/callbacks not removed** — event listeners registered but never de-registered keep the listener object alive.

```
import java.util.*;

class MemoryLeakExample {
    // BAD: static list grows forever, objects never removed
    static List<byte[]> leakList = new ArrayList<>();

    static void addData() {
        for (int i = 0; i < 1000; i++) {
            leakList.add(new byte[1024]);    // 1 KB per entry – never released
        }
    }

    // GOOD: use try-with-resources to guarantee resource closure
    static void readFile(String path) {
        try (java.io.BufferedReader br =
            new java.io.BufferedReader(new java.io.FileReader(path))) {
            System.out.println(br.readLine());
        } catch (Exception e) {
        }
    }
}
```

```

        e.printStackTrace();
    }
    // br is closed automatically – no leak
}

```

6.4 Key GC Concepts for Exam

Concept	Meaning
Reachable object	Object accessible through any live reference chain from a thread or static field
Eligible for GC	No live reference points to the object — it may be collected any time
System.gc()	Requests (does not force) the JVM to run GC; JVM may ignore it
finalize()	Called once by GC before reclaiming; deprecated Java 9+ — do not rely on it
Minor GC	Cleans the Young Generation (short-lived objects); fast and frequent
Major / Full GC	Cleans Old Generation; slow; triggers if heap fills up ('Stop-the-World')
Memory leak	Object is referenced but never used again — GC cannot collect it

7. Likely Semester-Exam Questions

Q1. What is the MVC design pattern? Explain its three components with a Java example.

MVC separates an application into Model (data + business logic), View (presentation), and Controller (intermediary). User interacts with View → Controller processes input → updates Model → Model notifies View → View re-renders. See Section 1 Student example. Benefits: separation of concerns, parallel development, easier testing, reusability.

Q2. What is the difference between 'commands as methods' and 'commands as objects'?

Commands as methods: actions embedded in method bodies, executed immediately when called — simple but cannot be deferred, logged, or undone. Commands as objects (Command Pattern): each action is encapsulated in an object implementing a Command interface with execute() and optionally undo(). An Invoker calls execute() without knowing what the command does. Supports undo/redo, queuing, logging, macro recording.

Q3. What is an exception in Java? How does Java handle exceptions?

An exception is an event that disrupts normal program flow at runtime. Java handles it via try-catch-finally blocks. Code that may throw goes in try; specific handlers in catch; cleanup code in finally (always runs). Keywords: try, catch, finally, throw (manually throw), throws (declare checked exceptions in method signature).

Q4. Differentiate between checked and unchecked exceptions.

Checked: detected at compile time; the compiler forces you to handle or declare them (try-catch or throws); extend Exception; examples: IOException, SQLException. Unchecked: detected at runtime; compiler does not force handling; extend RuntimeException; examples: NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException.

Q5. How do you create a custom exception in Java?

Extend Exception (for checked) or RuntimeException (for unchecked). Define constructor(s) that call super(message). Optionally add fields for extra context (like the shortage amount in InsufficientFundsException). Throw with 'throw new MyException("message")' and declare with 'throws MyException' in the method signature if checked. See Section 3.5.

Q6. What is the finally block? When does it NOT execute?

The finally block always executes after try and catch, whether or not an exception occurred — used for cleanup (closing files, releasing resources). Exception: finally does NOT execute if System.exit() is called inside the try/catch, or if the JVM crashes, or if the thread is killed externally.

Q7. What are packages in Java? What are their advantages?

A package is a namespace grouping related classes and interfaces, analogous to a folder. Advantages: avoids name collisions (java.util.Date vs java.sql.Date), enforces access control (default modifier = package-private), improves code organisation and reusability. Declared with 'package pkg.name;' as first statement; imported with 'import pkg.name.*;'.

Q8. What is the difference between Stack and Heap memory in Java?

Stack: stores method call frames, local variables, and references; LIFO order; per-thread; fast; automatically freed when method returns. Heap: stores all objects (created with new) and instance variables; shared across all threads; managed by the Garbage Collector; larger but slower; objects persist until GC.

Q9. What is garbage collection in Java? How does an object become eligible for GC?

GC is an automatic JVM process that identifies and frees heap memory occupied by unreachable objects. An object becomes GC-eligible when no live reference points to it: its reference goes out of scope, is set to null, or is overwritten with another reference. GC can be requested (not forced) via `System.gc()`; `finalize()` is called before collection.

Q10. What is a memory leak in Java? How do you prevent it?

A memory leak occurs when objects are referenced but never used again — the GC cannot collect them because a live reference still exists. Prevention: use try-with-resources for I/O objects, remove objects from static collections when done, de-register event listeners, prefer static nested classes over non-static inner classes to avoid implicit outer reference.

End of Unit 4 notes. Sources: Code World / arnab7070 (MVC, Command as object, memory management), GFG MVC Design Pattern (component communication, advantages), GFG Exception Handling (hierarchy, keywords, custom exceptions), GFG Packages and Access Modifiers (package creation, static imports, access table). Static members, nested classes, enumerations, GC internals, and exam Q&A written for this syllabus.